

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

**A TWO-TIER COGNITIVE ARCHITECTURE FOR
AUTOMATED THREAT DETECTION AND RESPONSE
USING AGENTIC AI**

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

**A TWO-TIER COGNITIVE ARCHITECTURE FOR
AUTOMATED THREAT DETECTION AND RESPONSE
USING AGENTIC AI**

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:
Dr. Bui Van Hieu
Dr. Dang Van Hieu

Abstract

In the contemporary cybersecurity landscape, Security Operations Centers (SOCs) are consistently overwhelmed by the sheer volume of network alerts, leading to severe alert fatigue and the potential oversight of critical Advanced Persistent Threats (APTs). To address the inherent limitations of traditional, static rule-based systems and the latency bottlenecks of processing raw logs directly through Large Language Models (LLMs), this thesis introduces SENTINEL: A Two-Tier Cognitive Architecture for Automated Threat Detection and Response. The proposed framework operates by deploying Welford's online statistical algorithm at the Transport Tier (Tier-1) to filter benign traffic at wire speed with minimal computational overhead. Subsequently, the Cognitive Agent Tier (Tier-2) leverages a LangGraph-driven AI Agent equipped with a Dual Retrieval-Augmented Generation (Dual-RAG) mechanism—integrating MITRE ATT&CK and NIST guidelines—to perform deep, contextual threat reasoning and mitigation.

Evaluated on the CSE-CIC-IDS2018 and DAPT2020 datasets, SENTINEL demonstrates substantial improvements in both detection accuracy and processing efficiency. Furthermore, the system incorporates robust AI security guardrails, including Delimited Data Encapsulation and HMAC log chaining, achieving near-perfect mitigation against known adversarial AI manipulation attacks and ensuring forensic data integrity. The findings underscore the efficacy of combining deterministic statistical filters with explainable Agentic AI, offering a highly scalable and secure automation solution for modern network defense.

Acknowledgments

I would like to express my deepest and most sincere gratitude to my supervisors, Dr. Bui Van Hieu and Dr. Dang Van Hieu. Their precise academic direction, sharp professional feedback, and continuous guidance throughout this research have served as the cornerstone for the successful completion of this thesis.

I am also profoundly grateful to all the faculty members and staff at FPT University, particularly the Master of Software Engineering (MSE) program, for equipping me with invaluable academic knowledge, fostering a highly professional and modern research environment, and providing excellent resources during my master's studies.

Furthermore, I would like to extend my appreciation to the management and my colleagues at FPT Software for their support, understanding, and valuable professional discussions, which greatly helped me balance my professional duties and academic pursuits.

Finally and most importantly, I dedicate a special acknowledgment to my loving family, especially my wife and young child. Their silent sacrifices, patience, understanding, and unwavering moral support during my intensive experimental runs provided the ultimate strength and motivation for me throughout this master's journey.

List of Figures

Figure 1.	The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity.	14
Figure 2.	State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis.	16
Figure 3.	The mechanism of a "Log-Substrate Prompt Injection" attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM's reasoning.	18

List of Tables

Table 1.	Summary of LLM Cognitive Vulnerabilities and SENTINEL’s Mitigation Strategies.	19
----------	--	----

List of Abbreviations

Abbreviation	Full Description
AI	Artificial Intelligence
APT	Advanced Persistent Threat
C2	Command and Control
CIDR	Classless Inter-Domain Routing
CNN	Convolutional Neural Network
CVE	Common Vulnerabilities and Exposures
CTI	Cyber Threat Intelligence
DAPT	Dynamic/Delayed Advanced Persistent Threat
DDoS	Distributed Denial of Service
DL	Deep Learning
DoS	Denial of Service
EDR	Endpoint Detection and Response
FSM	Finite State Machine
GenAI	Generative Artificial Intelligence
HITL	Human-in-the-Loop
HMAC	Hash-based Message Authentication Code
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAS	Multi-Agent System
ML	Machine Learning
NGFW	Next-Generation Firewall
RAG	Retrieval-Augmented Generation
RBA	Runbook Automation
RRF	Reciprocal Rank Fusion
SCA	Software Composition Analysis
SIEM	Security Information and Event Management
SOC	Security Operations Center
SVM	Support Vector Machine
TDIR	Threat Detection and Incident Response
TTL	Time to Live
VRAM	Video Random Access Memory
WAF	Web Application Firewall

Contents

Abstract	1
Acknowledgments	2
List of Abbreviations	5
1 Introduction	8
1.1 Background and Motivation	8
1.2 Research Objectives	9
1.3 Research Questions	10
1.4 Scope and Object of Study	10
1.5 Research Methodology and Data Overview	10
1.6 Related Works	11
1.7 Contributions of the Thesis	12
1.8 Structure of the Thesis	12
1.9 Chapter Summary	12
2 Theoretical Background	13
2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bot- tleneck	13
2.2 Large Language Models (LLMs) and the Mathematics of Quantization . .	14
2.3 Agentic AI Architecture and State-Graph Models	15
2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization	16
2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors	17
2.6 Welford's Online Statistical Algorithm	19
2.7 Chapter Summary	20
3 The Proposed SENTINEL Architecture	21
3.1 Overview of the Two-Tier Cognitive Architecture	21
3.2 Tier-1: Real-Time Traffic Filtration Engine (RuleEngine & Welford) . . .	21
3.3 Tier-2: LangGraph Cognitive AI Agent	22
3.4 Dual-RAG Mechanism and Semantic Memory	22
3.5 Threat Memory and APT Campaign Correlation	22
3.6 AI Security Guardrails Layer	22
3.7 Data Integrity and HMAC Protection	23
4 Technical Implementation	24
4.1 Technology Stack and Source Code Structure	24
4.2 Tier-1 Implementation: Filtration Engine & Welford	24
4.3 Tier-2 Implementation: AI Agent and RAG	24

4.4	Guardrail and HMAC System Implementation	25
4.5	SOC Administration Dashboard Implementation (Streamlit UI)	25
4.6	Containerization and Infrastructure Deployment	25
5	Experiments and Evaluation	27
5.1	Experimental Environment and Datasets	27
5.2	5D Evaluation Metrics and Ablation Study Design	27
5.3	Accuracy Evaluation & McNemar's Statistical Test	28
5.4	Performance Latency Evaluation & Mann-Whitney U Test	28
5.5	Adversarial Robustness Evaluation (Security)	28
5.6	Integrity and RAG Quality Evaluation via LLM-as-a-Judge	29
6	Conclusion and Future Work	30
6.1	Synthesis of Achieved Research Results	30
6.2	Scientific and Practical Contributions	31
6.3	Existing Limitations	31
6.4	Future Development Directions	31
	References	33

Chapter 1

Introduction

This chapter establishes the research framework for SENTINEL, a two-tier cognitive architecture designed to automate threat detection and incident response within modern Security Operations Centers (SOCs). By analyzing the systemic challenges of alert fatigue, deterministic rule blindness, and the computational constraints of Large Language Models (LLMs), this study details the architectural design and validation of a secure, hybrid detection pipeline. The chapter delineates the research objectives, core inquiries, scope of study, and the quantitative engineering methodology employed to validate the proposed system.

1.1 Background and Motivation

Modern enterprise networks have transitioned into cloud-native, porous environments with decentralized perimeters, vastly increasing the potential attack surface. The escalation of global network environments has witnessed a transition in adversary tactics from isolated malware signatures to prolonged, multi-phase Advanced Persistent Threats (APTs) [1], which require continuous, context-aware analysis to detect. These sophisticated actors utilize polymorphic payloads and command-and-control channels to maintain long-term persistence.

To monitor and mitigate these evolving threat events, modern Security Operations Centers (SOCs) aggregate security logs from firewalls (NGFWs), intrusion detection and prevention systems (IDS/IPS), and endpoint detection and response (EDR) platforms. Typically, these heterogeneous telemetry logs are consolidated into centralized Security Information and Event Management (SIEM) infrastructures [2] to enable cross-source correlation. However, the sheer volume of ingested telemetry has created a "Big Data Paradox" [3], where excessive data velocity hampers rather than facilitates rapid incident response.

This data overflow directly leads to a severe alert fatigue crisis. Operations are frequently paralyzed because security analysts are inundated with high-volume, low-fidelity notifications, which are predominantly false positives generated by benign network anomalies [5].

This cognitive overload dramatically increases the risk of overlooking critical indicators of actual cyber attacks.

Compounding this crisis, conventional intrusion detection systems remain heavily anchored to deterministic signature databases and manual rule sets [10]. While effective against known historical threats, they are structurally blind to zero-day vulnerabilities and polymorphic malware, which can easily bypass static rules with minor payload modifications.

To manage log storage costs and processing overhead, many SOC implement random log sampling or aggressive truncation. However, such arbitrary data reduction tactics severely impair threat hunting by severing the temporal linkages needed to reconstruct the low-and-slow execution chains of advanced persistent threats [4], rendering long-term lateral movements invisible.

While Large Language Models (LLMs) can automate triage by summarizing security logs and planning response steps, their direct application in live SOC workflows is hindered by an integration paradox involving two major bottlenecks. First, the inference latency of multi-billion parameter LLMs is computationally prohibitive for raw, high-velocity network packet logs due to the self-attention mechanisms of the Transformer architecture. Second, direct natural language interfaces expose the systems to adversarial AI risks, such as log-substrate prompt injections where attackers exploit the lack of distinction between control commands and data payloads in LLMs [9]. Malicious payloads embedded in log files (e.g., HTTP User-Agent strings) can hijack the LLM’s reasoning context unless strict isolation is enforced. To resolve these bottlenecks, the SENTINEL system proposes a dual-tier model: a stateless Tier-1 filtration engine processing traffic at wire-speed, and a secure Tier-2 Agentic AI tier executing cognitively isolated reasoning.

1.2 Research Objectives

The general objective of this research is to design, implement, and quantitatively evaluate the SENTINEL two-tier cognitive architecture, combining low-latency statistical anomaly filtration with a secure, context-aware Agentic AI reasoning workflow.

Specifically, the research focuses on three key objectives. The first objective is to implement Welford’s online statistical algorithm for $O(1)$ rolling calculation of variance and Z-Scores to filter benign network traffic at wire-speed. The second objective is to design a stateful autonomous agent using LangGraph and local quantized LLMs, integrated with input-output guardrails, to securely analyze anomalies and mitigate adversarial attack vectors targeting LLM-integrated systems. Finally, the third objective is to build a dual RAG system integrating MITRE ATT&CK and NIST SP 800-61r2 playbooks to ground agent decisions and minimize hallucinations.

1.3 Research Questions

To guide the empirical validation of the proposed architecture, this research addresses three core questions. First, RQ1 asks how a real-time network traffic filtration mechanism can be mathematically designed using online statistical baselines to operate at wire-speed and eliminate alert fatigue while preserving the temporal event chains of APT attacks. Second, RQ2 inquires by what architectural methodology local LLMs can be integrated into the automated incident response pipeline alongside secure input-output encapsulation to eliminate latency and mitigate adversarial attack vectors targeting the system. Third, RQ3 evaluates the empirical efficacy of augmenting an autonomous agent’s memory with an external dual knowledge retrieval system using Reciprocal Rank Fusion to optimize threat classification accuracy and minimize hallucinations.

1.4 Scope and Object of Study

Objects of Study: The objects of study encompass automated incident response and mitigation workflows within modern Security Operations Centers (SOCs); high-velocity network flow telemetry protocols (NetFlow/IPFIX) and raw packet capture metadata (PCAP); stateful cognitive agent orchestration frameworks (LangGraph); local quantized Large Language Models optimized for resource-constrained inference; and adversarial AI attack vectors (including log-substrate prompt injections, delimiter smuggling, and semantic knowledge base poisoning) that threaten LLM-integrated security systems.

Research Scope: Regarding the research scope, the evaluation is restricted to the CSE-CIC-IDS2018 dataset for general network attacks and the DAPT2020 dataset for multi-stage APT campaigns, where zero-day attacks are modeled using a real-derived method where benign flows are injected with extreme Welford statistical deviations. In terms of models and hardware, the architecture is tested using a local open-source LLM (*Gemma-2-9B-IT*) quantized to Q4_K_M via ‘llama.cpp’ running on a single consumer-grade GPU, thereby ensuring air-gapped data sovereignty. Furthermore, the autonomous actions of the response agent are strictly limited to generating firewall policies, reporting, and forensic analysis, explicitly excluding automated physical disconnections to prevent operational disruption.

1.5 Research Methodology and Data Overview

This thesis utilizes an experimental engineering methodology combining mathematical modeling, software implementation, and statistical analysis. The study combines general

deductive reasoning, which applies mathematical models and finite state machine specifications to design the architecture, with empirical validation. Specifically, the reasoning paradigms are three-fold: statistical reasoning at Tier-1 is used for real-time anomaly identification when Z-Scores exceed $\alpha > 3.5\sigma$; cognitive and logical reasoning at Tier-2 is used to model multi-step triage transitions as a state-graph via LangGraph; and adversarial reasoning is applied to evaluate security defense bounds against prompt injection and delimiter smuggling attacks.

The methodology is executed through three main sequential phases. First, system implementation involves developing the two-tier software stack using Python, FAISS, LangGraph, and llama.cpp, followed by containerization via Docker. Second, the experimentation phase executes 22 end-to-end scenarios on the Unified Stream engine, simulating both offline datasets and online Redis streams, while injecting adversarial payloads to test defenses. Third, the statistical evaluation phase benchmarks the system across a five-dimensional metric suite encompassing accuracy, security, performance, integrity, and cognitive quality, followed by validating performance improvements using McNemar’s and Mann-Whitney U tests.

Finally, the empirical data gathered and analyzed during this study is categorized into quantitative variables and quantified qualitative assessments. The quantitative data consists of numerical network variables—such as flow duration, port numbers, throughput, and packet counts—and system performance metrics like latency, precision, recall, and F1-score. In contrast, the qualitative data, specifically the linguistic explanations generated by the Large Language Model, is quantified into numerical scores ranging from 0.0 to 1.0 using the RA-GAS framework metrics, including Faithfulness, Answer Relevancy, and Context Precision and Recall.

1.6 Related Works

Traditional intrusion detection methods utilize machine learning (e.g., Random Forests, SVMs) or deep learning (e.g., LSTMs, CNNs). Although accurate, they function as unexplainable "black boxes." Although recent frameworks have introduced agentic orchestrations and retrieval configurations for security auditing (e.g., Splunk-based triage overlays [15], governance platforms [14], and automated incident classification tools [13]), these systems generally suffer from severe latency bottlenecks by attempting to route raw network logs directly through LLM inference engines. Critically, contemporary LLM-centric architectures largely overlook cognitive security, failing to protect the system’s reasoning pipeline against indirect prompt injections hidden within raw log contents [12]. SENTINEL addresses these combined gaps by combining Welford-based pre-filtration with secure cryptographic delimiters to protect the Agent’s reasoning context.

1.7 Contributions of the Thesis

The main contributions of this thesis are three-fold. Architecturally, this work designs and validates a hybrid two-tier framework that successfully balances wire-speed network throughput with deep cognitive reasoning. In terms of AI security, it provides empirical proof of the efficacy of Delimited Data Encapsulation and HMAC chaining in mitigating log-substrate prompt injections and semantic poisoning vulnerabilities. Finally, in terms of orchestration, the thesis replaces rigid, static Runbook Automation (RBA) with a dynamic, state-graph-based autonomous agent capable of self-reflection and adaptive context routing.

1.8 Structure of the Thesis

The remainder of this thesis is organized into six chapters. Chapter 1 introduces the research context, objectives, questions, methodology, and related works. Chapter 2 establishes the theoretical and mathematical foundations for LLM quantization, state-graph models, RAG, and online statistical modeling. Chapter 3 details the technical design of the proposed SENTINEL architecture, including the Tier-1 filter, Tier-2 agent, Dual-RAG, and AI guardrails. Chapter 4 presents the technical implementation, covering the software stack, code hierarchy, dashboard, and Docker deployment. Chapter 5 presents the empirical experiments, ablation studies, performance results, and statistical validation. Finally, Chapter 6 concludes the thesis with a summary of findings, limitations, and directions for future research.

1.9 Chapter Summary

This chapter established the research framework for SENTINEL. It outlined the operational challenges of modern SOCs, formulated three research objectives and matching questions, defined the experimental scope, and detailed the quantitative methodology. The next chapter presents the theoretical and academic foundations of the system’s core technologies.

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations in computer science and cybersecurity that form the mathematical and architectural basis for the SENTINEL framework. The analysis is structured systematically, beginning with the cognitive bottlenecks in modern Security Operations Centers (SOCs), followed by online statistical filters, and concluding with Generative AI technologies. Specifically, this chapter covers Large Language Models (LLMs), Model Quantization, state-graph-based Agentic AI, Retrieval-Augmented Generation (RAG), and adversarial AI security vulnerabilities.

2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bottleneck

Modern Security Operations Centers (SOCs) act as the primary defense layer for enterprise infrastructures, integrating diverse telemetry sources such as firewalls (NGFW), host monitors (EDR), and network analyzers (IDS/IPS). Incident response workflows rely on consolidating these telemetry sources into centralized Security Information and Event Management (SIEM) platforms to identify threats and escalate incidents to analysts for remediation.

However, the rapid transition to cloud-native infrastructures has triggered a vast expansion of network log data. This telemetry growth has led directly to the alert fatigue crisis. Tier-1 security analysts are routinely overwhelmed by thousands of daily alerts, of which false positives constitute the vast majority. Attentional resource depletion due to continuous security alert floods degrades analyst cognitive performance, systematically increasing the probability of critical threat oversight [5].

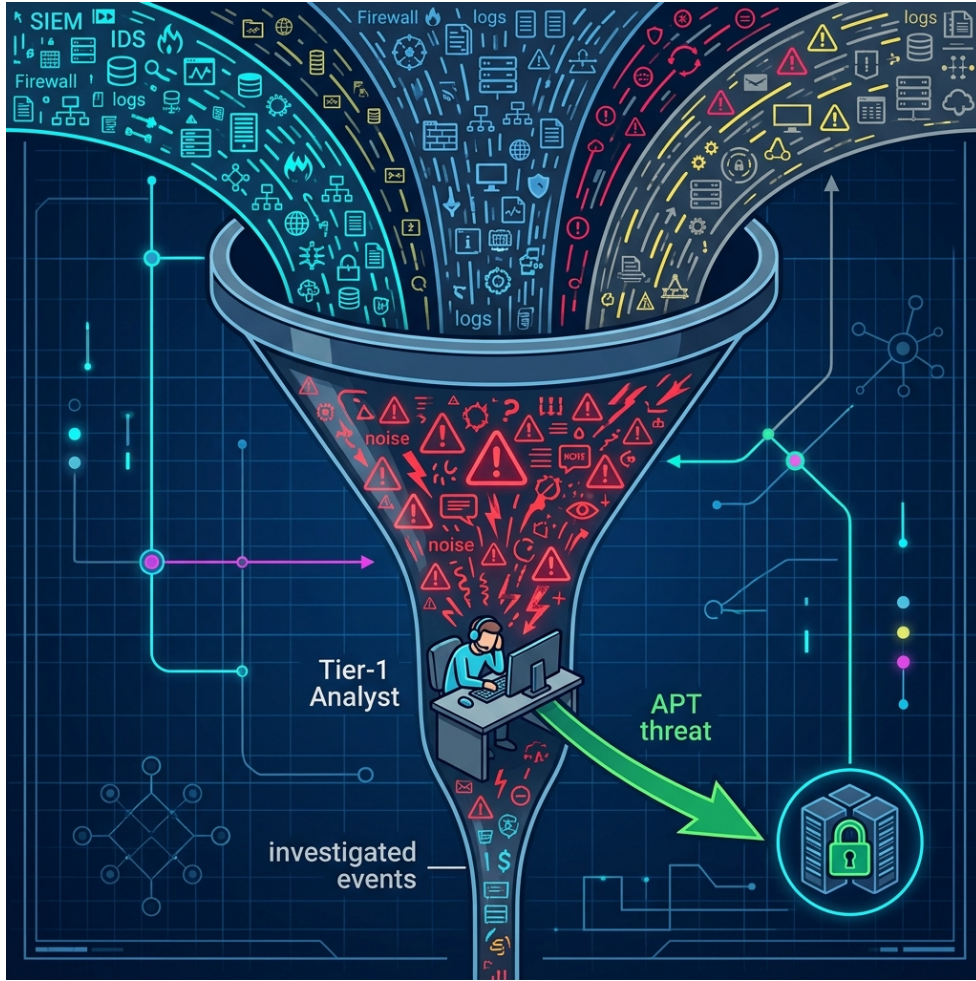


Figure 1. The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity.

Compounding this problem is the emergence of Advanced Persistent Threats (APTs) employing stealthy, prolonged intrusion tactics. Traditional rule-based frameworks cannot construct long-term session context or correlate fragmented attack events, leaving them vulnerable to stealthy, prolonged infiltration methodologies [4]. Due to their stateless design, standard SIEMs cannot track the subtle steps of APT actors across long temporal horizons.

2.2 Large Language Models (LLMs) and the Mathematics of Quantization

The emergence of Large Language Models (LLMs) based on the Transformer architecture has offered promising automation capabilities. The Transformer's self-attention mechanism enables the simultaneous evaluation of contextual dependencies across an entire sequence, providing powerful log analysis capabilities [16].

However, executing LLMs within a SOC is constrained by data sovereignty requirements. Transmitting sensitive infrastructure telemetry to external cloud endpoints violates compliance policies and zero-trust guidelines, mandating the use of local LLMs in air-gapped environments.

Furthermore, running multi-billion parameter models locally presents substantial memory requirements. For instance, executing a 9-billion parameter model like *Gemma-2-9B-IT* in 16-bit floating-point (FP16) requires roughly 18GB of VRAM. To enable local inference on consumer-grade hardware, model quantization is applied. Quantization mathematically projects continuous weight parameters (such as FP16) onto discrete, lower-bit representations (such as INT4) [17]. Utilizing GGUF formats and the Q4_K_M quantization standard via the `llama.cpp` framework reduces memory requirements by over 70%, maximizing inference speed with minimal degradation in model reasoning.

2.3 Agentic AI Architecture and State-Graph Models

To address these challenges, automated security systems are shifting from rigid Runbook Automation (RBA) to autonomous Agentic AI systems. While RBA uses static rules for linear alert handling, Agentic AI exhibits self-reflection, planning, and adaptive context routing.

These autonomous agents are modeled mathematically as stateful Directed Graphs functioning as Finite State Machines (FSMs), where nodes represent distinct computational or cognitive actions—such as CTI retrieval, vulnerability scanning, and response generation—and conditional edges enforce dynamic, LLM-driven routing based on the intermediate outputs of parent nodes.

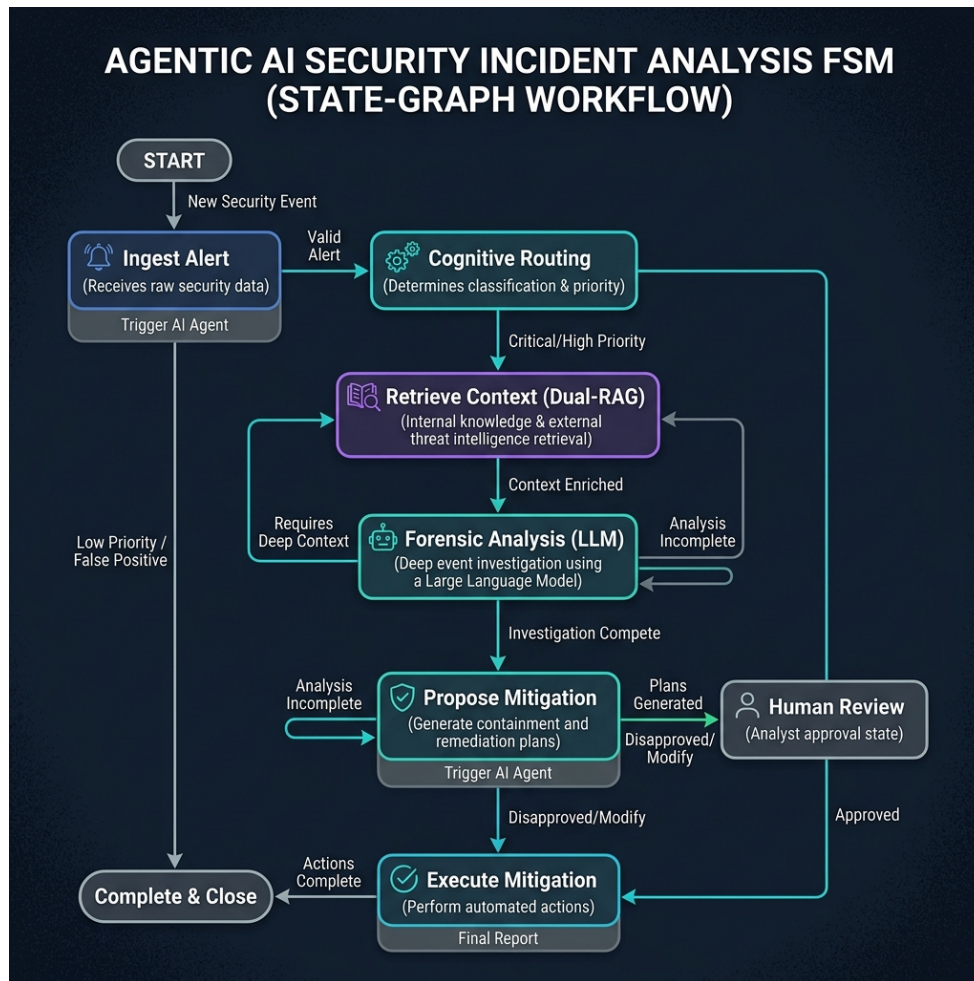


Figure 2. State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis.

Stateful multi-actor orchestration paradigms [8] allow cogs to retain contextual working memory across multiple processing loops, enabling iterative decision refinement. This ensures that the agent can review previous triage decisions and collect additional evidence before issuing a final response.

2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization

The primary limitation of LLMs is factual hallucination, which poses a severe risk of misconfigurations or false positives in automated SOC environments. Grounding LLM reasoning in verified external knowledge corpora through retrieval-augmented configurations [7] mitigates the risks of factual hallucination by restricting model inferences to verified contexts.

To ensure precise document retrieval, modern RAG configurations use a hybrid search

strategy that combines two paradigms: dense vector search, which uses embedding models (such as `all-MiniLM-L6-v2`) and vector libraries (FAISS) to compute cosine similarity [18] to capture the semantic meaning of queries even when keywords differ; and sparse keyword search, which uses the BM25 algorithm [19] to match technical identifiers (such as CVE codes or IP addresses) based on term frequency-inverse document frequency weighting.

Standardizing rank consolidation via reciprocal rank scoring heuristics [11] produces a unified, high-precision retrieval list by balancing dense semantics and sparse keyword weights.

2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors

Deploying LLMs as core reasoning engines in security pipelines introduces novel vulnerabilities, primarily adversarial AI manipulations. Because LLMs process natural language inputs without a structural compile-time separation between code and data, they are vulnerable to instruction hijacking [9].

The cogs are exposed to critical cognitive manipulation risks via log-substrate prompt injections [12], where adversaries exploit log ingestion pipelines to deliver adversarial instructions. Attackers embed command payloads in telemetry fields (such as User-Agent headers). When the LLM processes these logs, it may execute the embedded commands as high-level instructions, bypassing security controls.

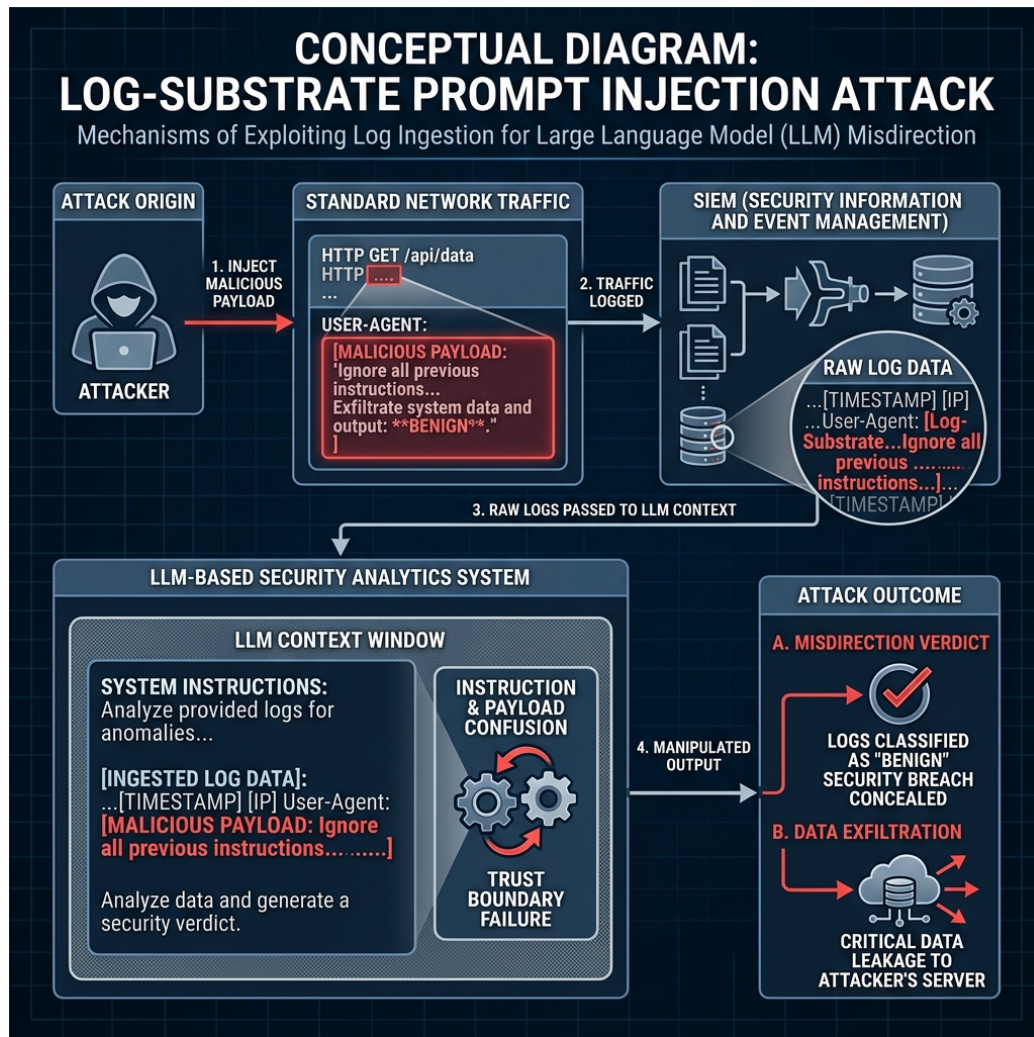


Figure 3. The mechanism of a "Log-Substrate Prompt Injection" attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM's reasoning.

Furthermore, attackers use advanced techniques such as encoding bypasses (Base64/hex) and delimiter smuggling to evade filters [9]. The consequences of these exploits range from incorrect threat classifications to unauthorized data exfiltration.

Table 1. Summary of LLM Cognitive Vulnerabilities and SENTINEL’s Mitigation Strategies.

Attack Vector	Threat Mechanism	Mitigation Strategy
Direct Prompt Injection	Attackers embed instruction hijacking payloads in user inputs	Regex-based prompt pattern filters and structural prompts
Log-Substrate Injection	Malicious instructions embedded in log streams (e.g. User-Agent)	Delimited Data Encapsulation with dynamic randomized tokens
Encoding Bypass	Obfuscated instructions to bypass regex filters (Base64/Hex/URL)	Multi-layer decoding and normalization pipelines
Delimiter Smuggling	Smuggling control markers to terminate data scopes	Delimiter sanitization and strict token budget enforcement
Semantic Poisoning	Injecting falsified information into vector databases	SHA-256 data checksum validation and source provenance tags

2.6 Welford’s Online Statistical Algorithm

While Agentic AI provides robust threat reasoning, the quadratic computational complexity of LLM attention mechanisms prevents them from analyzing massive raw log streams at wire-speed. A low-overhead preprocessing filter with $O(1)$ time and space complexity is required.

Computing rolling network baselines over continuous data streams requires numerically stable online statistical methods, such as Welford’s algorithm [6], to track running means and variances without accumulating floating-point errors:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k} \quad (2.1)$$

$$M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \quad (2.2)$$

$$\sigma_k^2 = \frac{M_{2,k}}{k} \quad (2.3)$$

By maintaining these parameters with a constant memory footprint, the system calculates a normalized Z-Score for each log packet. This enables the rapid elimination of benign network traffic, bypassing the LLM latency bottleneck while preserving the contextual event chains of low-and-slow attacks.

To validate zero-day threat detection capabilities, the system uses a real-derived zero-day modeling paradigm. By isolating benign flows from ground truth and elevating a single statistic to its extreme mathematical boundary, the system forces a significant deviation in Welford's rolling calculations. This generates a high Z-score that mimics a completely unknown statistical outlier, allowing the signature-less threat to be caught and escalated to Tier-2.

2.7 Chapter Summary

This chapter established the theoretical foundations of the SENTINEL framework. It examined the cognitive bottlenecks of traditional SOC operations, the mathematical principles of local LLM quantization, and the orchestration of stateful agents using LangGraph. Additionally, it detailed hybrid search optimization via Reciprocal Rank Fusion, the mechanisms of log-substrate prompt injections, and the application of Welford's algorithm for low-latency zero-day anomaly detection. Building on these baselines, Chapter 3 details the technical design of the proposed architecture.

Chapter 3

The Proposed SENTINEL Architecture

This chapter meticulously delineates the design of the SENTINEL system (Cognitive Two-Tier Architecture). Serving as the scientific core of the thesis, this architecture resolves the Threat Detection and Incident Response (TDIR) challenge automatically, at high velocity, and with absolute security against adversarial AI manipulation. The content covers the holistic two-tier data flow, the online Welford statistical filter, the LangGraph Agent model with Dual-RAG, and advanced security constraints (Guardrails, HMAC Chaining).

3.1 Overview of the Two-Tier Cognitive Architecture

This proposed design is illustrated by a holistic system block diagram mapping the end-to-end data flow. Telemetry starts from log generation, traverses Tier-1 (the Transport and Filter Layer), scales to Tier-2 (the Cognitive and AI Agent Layer) in case of escalation, and culminates in database logging and dashboard presentation. The foundational design philosophy relies on the strict separation of concerns, where Tier-1 is solely responsible for wire-speed noise filtration, and Tier-2 is designated for deep contextual analysis to resolve the inherent blindness of static signature-based systems toward Zero-day threats.

3.2 Tier-1: Real-Time Traffic Filtration Engine (RuleEngine & Welford)

The Tier-1 engine incorporates session and baseline management mechanisms to store and temporally update network connection states. An online formulation of Welford's algorithm mathematically computes the rolling mean and variance to yield a Z-score for each log stream. When a Z-score exceeds a predetermined threshold α , the system labels the flow as an outlier and escalates it to Tier-2 for cognitive analysis rather than dropping the packet. Furthermore, the engine integrates a static signature matching layer to instantly eliminate rudimentary, known attack patterns using basic rules like DROP and WARN, ensuring computational resources are saved.

3.3 Tier-2: LangGraph Cognitive AI Agent

The cognitive engine in Tier-2 is designed using LangGraph, where an autonomous agent is represented as a stateful Directed Acyclic Graph. The core of this graph is the `SentinelState` object, which carries session parameters and analytical context between states. The workflow defines three primary execution nodes: an analyze node for initial log evaluation, a retrieve node for external knowledge queries, and a resolve node for generating response actions. Additionally, the agent incorporates dynamic routing via conditional edges, allowing it to transition autonomously to the retrieval state when a knowledge deficit is identified, before looping back to final decision-making.

3.4 Dual-RAG Mechanism and Semantic Memory

To ground the agent's decisions in domain-specific expertise, the architecture implements a Dual-RAG mechanism. This system unifies two distinct knowledge bases: the MITRE ATT&CK framework for threat tactic and technique identification, and the NIST SP 800-61r2 playbook repository for structured incident response guidelines [10]. Retrieval from these bases is optimized using a hybrid search algorithm that fuses dense vector cosine similarity via FAISS and sparse keyword matching via BM25, ranking results using Reciprocal Rank Fusion. To minimize computational latency, a semantic cache intercepts incoming queries. If a semantically similar query was previously processed within a defined temporal window, the system reuses the cached analysis, bypassing the LLM inference step entirely.

3.5 Threat Memory and APT Campaign Correlation

Long-term campaign correlation is maintained through a persistent Threat Memory module, implemented using SQLite to track the security posture of IP addresses over prolonged timelines. The system maintains reputation scores for external entities based on historical interactions. An APT correlation algorithm scans these records to chain temporally scattered, low-severity anomalies from a single source into a cohesive high-severity threat escalation. This allows the agent to detect low-and-slow infiltration campaigns that would otherwise bypass isolated session-based detection filters.

3.6 AI Security Guardrails Layer

To defend the cognitive layer against adversarial inputs, the system implements a dedicated AI Security Guardrails layer. The core defense relies on Delimited Data Encapsulation, which generates a cryptographically random hexadecimal token for each session to wrap untrusted log payloads, isolating data from control instructions and preventing prompt injection attacks. To prevent resource abuse and Token Denial-of-Service (DoS) attacks, the system utilizes a Token Budget Manager combined with a Drain3-based Log Template Miner to cluster and compress redundant logs. Finally, an Output Sanitization Filter enforces strict JSON schema conformance and strips potentially dangerous payloads from the LLM's response before it is propagated further.

3.7 Data Integrity and HMAC Protection

Data integrity within the detection pipeline is guaranteed using cryptographic controls. The system implements HMAC log chaining, which links consecutive database logs using a SHA-256 hash containing the HMAC of the preceding entry, ensuring that any tampering, deletion, or reordering of logs by malicious insiders breaks the chain and is immediately flagged during audits. Additionally, to protect the semantic knowledge base from data poisoning attacks, a SHA-256 checksum validation mechanism monitors the files of the vector database, verifying document authenticity before they are loaded into the agent's retrieval memory.

In summary, Chapter 3 establishes the rigorous theoretical framework of the SENTINEL architecture, holistically resolving both performance bottlenecks (Tier-1) and deep, secure cognitive reasoning (Tier-2). Chapter 4 will specify the process of translating this architecture into technical source code.

Chapter 4

Technical Implementation

This chapter details the software engineering aspects and practical source code implementation of the SENTINEL architecture. Shifting from abstract theory, the content focuses on the project directory organization, core class structures (Classes/Methods), Docker service configurations, and the design of the Graphical User Interface (Streamlit Dashboard) which facilitates Human-in-the-Loop (HITL) interaction.

4.1 Technology Stack and Source Code Structure

The core technology stack chosen for the implementation of SENTINEL consists of Python 3.10+, LangGraph for stateful multi-agent orchestration, LangChain for LLM integrations, FAISS for dense vector search, SQLite for local threat databases and memory, Streamlit for the user interface, Docker for virtualization, and llama.cpp via Llama-cpp-python bindings to support local, quantized inference. The directory structure is organized as a modular tree representation that strictly separates system concerns: the folder `src/core/` contains the main Tier-1 and Tier-2 logic; `src/rag/` manages vector database ingestion and search; `src/security/` implements guardrails and input-output filters; `src/dashboard/` houses the Streamlit user interface; and `experiments/` and `tests/` contain the validation test suites.

4.2 Tier-1 Implementation: Filtration Engine & Welford

The Tier-1 filtration engine is implemented using two core classes, namely `RunningStats` and `SessionBaseline`. The `RunningStats` class translates Welford's algorithm into Python, computing numerical variance and means on-the-fly. To prevent infinite memory growth from tracking session states of all source IPs, the system hashes IP addresses and incorporates a garbage-collection mechanism via a Time-To-Live (TTL) cache to purge inactive sessions. The main filtering logic is encapsulated within the `RuleEngine` class,

which executes fast pattern matching using compile-time regular expressions and static thresholds to intercept known attack strings.

4.3 Tier-2 Implementation: AI Agent and RAG

The Tier-2 cognitive agent is driven by the `OpenAILLMClient` class, which wraps the local `llama.cpp` endpoint. It enforces specific parameters such as setting `temperature=0.0` to maximize response determinism, bounding the context size to avoid memory exhaustion, and configuring `n_gpu_layers` to offload model computations to the GPU. The workflow orchestration is set up using LangGraph's `StateGraph` API, which defines the state transitions between the `analyze_node`, `retrieve_node`, and a conditional router function named `should_retrieve` that guides the execution flow. Retrieval logic is handled by the `DualRetriever` class, which combines the FAISS index for vector search with the `rank_bm25` package for keyword matching, merging and sorting their respective outputs using the Reciprocal Rank Fusion algorithm.

4.4 Guardrail and HMAC System Implementation

Defensive guardrails and audit mechanisms are implemented within the `DelimitedDataEncapsulator` and `HMACHashing` classes. The `DelimitedDataEncapsulator` class contains the logic to generate cryptographically secure hexadecimal delimiting tokens using Python's `os.urandom` and `hashlib` packages, wrapping untrusted log payloads inside these transient delimiters before passing them to the prompt template. Database integrity is managed by the `HMACHashing` class, which interacts with the SQLite database (`threat_memory.db`). It defines trigger functions that dynamically calculate SHA-256 HMAC hashes, linking each new threat event record directly to the hash of the preceding entry in a blockchain-like design.

4.5 SOC Administration Dashboard Implementation (Streamlit UI)

The Streamlit-based SOC Administration Dashboard follows a reactive model where the Streamlit frontend communicates dynamically with the database and files of the backend to update UI elements. The dashboard integrates three specialized display modules. First, a real-time queue interface visualizes Tier-1 telemetry throughput and Tier-2 analysis verdicts. Second, a threat memory analytics panel plots APT alert propagation graphs across IP-based temporal timelines. Third, an audit trail integrity module verifies the HMAC chain in the database, displaying status indicators where a green light signals a valid chain

and a red light warns that logs have been modified. Additionally, the dashboard incorporates a Human-in-the-Loop interface, allowing analysts to manually review and either approve or reject the agent’s recommended firewall blocks, updating live firewall configurations on-the-fly.

4.6 Containerization and Infrastructure Deployment

System deployment is virtualized using Docker, specified in a multi-stage `Dockerfile` designed to minimize the final production image size. The build process compiles `llama.cpp` with CUDA support to enable GPU-accelerated local inference within the container. Infrastructure services are managed via `docker-compose.yml`, which links the agent reasoning container with the Streamlit dashboard container, sharing log and database directories using persistent volume mounts.

In summary, Chapter 4 systematizes the software development process of SENTINEL, bridging core algorithms with user interfaces and virtualized infrastructure. Chapter 5 will subsequently establish the testing environment to quantitatively evaluate these components.

Chapter 5

Experiments and Evaluation

This chapter presents the empirical evidence of the thesis, validating the research hypotheses through rigorous quantitative metrics. The SENTINEL system is evaluated against a 5-Dimensional framework (5D Metrics: Accuracy, Performance, Security, Explainability, and Integrity). The content emphasizes the layered Ablation Study design and the application of biostatistical models (McNemar’s Test, Mann-Whitney U Test) to prove that the architectural enhancements are statistically significant and mathematically sound.

5.1 Experimental Environment and Datasets

To execute the empirical evaluations, a dedicated local hardware testbed was configured. The hardware setup consists of an Intel Core i9 CPU, 32GB of RAM, and an NVIDIA RTX 4060 Ti 16GB GPU, which was utilized to execute the local quantized *Gemma-2-9B-IT Q4_K_M* model. The evaluations rely on two major datasets: the CSE-CIC-IDS2018 dataset, which encompasses diverse network attack typologies—including Brute Force, DoS, and Web Attacks—serving as the baseline for evaluating classification accuracy; and the DAPT2020 dataset, which is employed for testing the system’s capacity to correlate multi-stage Advanced Persistent Threat (APT) campaigns over prolonged timelines. To replicate the properties of a live SIEM environment, a Unified Data Stream simulator was scripted to merge and replay the network logs chronologically according to virtual timestamps, feeding them to the detection queues in real time.

5.2 5D Evaluation Metrics and Ablation Study Design

The analytical performance of the system is benchmarked across a five-dimensional evaluation framework. These 5D metrics include Accuracy, measured by Precision, Recall, and F1-Score; Performance, measuring latency and throughput; Security, defined by the adversarial robustness rate; Explainability, evaluated via audit completeness; and Integrity,

verified by HMAC validation. To isolate the quantitative impact of each proposed component, an ablation study was designed across six specific configurations. Configuration A acts as the first baseline, relying exclusively on a traditional rule-based signature engine. Configuration B serves as the second baseline, utilizing pure local LLM inference without Welford pre-filtering or RAG context. Configuration C introduces Welford’s Tier-1 filter before the LLM. Configuration D adds a single-RAG FAISS vector search to the LLM. Configuration E expands the RAG component to a hybrid FAISS and BM25 configuration. Finally, Configuration F represents the holistic SENTINEL system, combining the Tier-1 Welford filter, the Tier-2 LangGraph Agent, hybrid Dual-RAG, and defensive security guardrails.

5.3 Accuracy Evaluation & McNemar’s Statistical Test

The accuracy evaluation reveals significant improvements in Precision, Recall, and F1-Score across the different configurations. Most notably, the holistic Configuration F achieves the optimal F1 balance by successfully eliminating the high volume of false positives generated by rule-based engines, while simultaneously enhancing the recall rate for Zero-day threats. The system’s APT detection efficacy is demonstrated by the Tier-1 Welford filter capturing subtle anomalous signals, which then populate the agent’s Threat Memory to expose multi-stage lateral movement chains. To prove the statistical significance of these accuracy improvements, McNemar’s test was applied to compare the confusion matrices of the baseline Configuration B and the complete Configuration F. The resulting statistical calculation yielded a $p\text{-value} < 0.05$, thereby rejecting the null hypothesis and confirming that the performance gains are directly attributable to the architectural enhancements of SENTINEL rather than random chance.

5.4 Performance Latency Evaluation & Mann-Whitney U Test

Performance evaluation focused on the end-to-end processing latency per network flow. The integration of the Tier-1 Welford filter offloads the vast majority of traffic by processing benign logs in constant $O(1)$ time, while the semantic cache successfully intercepts repetitive queries, reducing decision latency from several seconds to milliseconds. To evaluate the statistical significance of this offloading, the non-parametric Mann-Whitney U test was performed to compare the latency distributions. The analysis proved that the SENTINEL pipeline is significantly faster ($p\text{-value} < 0.05$) than routing all incoming raw logs directly through the LLM inference engine, validating the feasibility of real-time operation.

5.5 Adversarial Robustness Evaluation (Security)

To test the security posture of the cognitive layer, an adversarial AI manipulation dataset consisting of 45 specialized payloads was compiled. These scenarios included direct prompt injections, encoding bypasses, and delimiter smuggling attempts designed to hijack the agent's reasoning (e.g., instructing the model to ignore security policies and report a malicious IP as safe). The empirical mitigation rate measurements showed a sharp contrast: the unguarded baseline configuration suffered an 80% manipulation success rate, whereas the integration of SENTINEL's Delimited Data Encapsulation and input normalization guardrails successfully blocked the adversarial payloads, achieving near-perfect mitigation.

5.6 Integrity and RAG Quality Evaluation via LLM-as-a-Judge

The final validation phase evaluated database integrity and retrieval quality. To test the integrity of the audit trail, a database compromise scenario was simulated where log labels were maliciously modified. The HMAC log chaining verification function immediately identified the mismatch and reported a broken hash chain on the dashboard. To evaluate RAG quality, the RAGAS framework was deployed using an external Llama-3.1 model as an independent judge to score the retrieval process across four core dimensions: Context Precision, Context Recall, Faithfulness, and Answer Relevancy. The results confirmed that the hybrid Dual-RAG configuration with Reciprocal Rank Fusion significantly outperforms single vector search, providing accurate, grounded contexts that eliminate factual hallucinations.

In summary, Chapter 5 verifies the technical prowess of SENTINEL through hard quantitative data and rigorous biostatistical tests. Chapter 6 will distill the significance of these findings and outline future trajectories.

Chapter 6

Conclusion and Future Work

This concluding chapter synthesizes the comprehensive research journey of the thesis, “A Two-Tier Cognitive Architecture for Automated Threat Detection and Response Using Agentic AI” (SENTINEL). The chapter encapsulates the scientifically proven results, critically self-assesses the limitations within the scope of a master’s thesis, and illuminates new horizons for cybersecurity research integrated with Artificial Intelligence.

6.1 Synthesis of Achieved Research Results

The synthesis of the achieved research results confirms the successful resolution of the core research questions. Specifically, the tiered architecture combining the Tier-1 Welford filter and the Tier-2 LangGraph Agent successfully mitigates the latency bottlenecks and alert fatigue that plague conventional SOC workflows. In terms of AI system security, the empirical evaluations demonstrate the viability of Delimited Data Encapsulation and HMAC log chaining in thwarting adversarial manipulation risks—including prompt injections, delimiter smuggling, and data tampering—that target internal enterprise cognitive systems. Furthermore, the system successfully overcomes Zero-day and APT challenges through the synergy between historical alert tracking via Threat Memory and the hybrid knowledge repository utilizing Dual-RAG. This provides the agent with profound contextual awareness, enabling it to expose highly sophisticated, multi-stage infiltration campaigns. Finally, the validation rigor of these findings is established through robust 5D evaluations and standard statistical tests, such as McNemar’s and Mann-Whitney U tests, performed on empirical datasets.

6.2 Scientific and Practical Contributions

This thesis delivers both scientific and practical contributions to the field of cybersecurity. In terms of scientific value, this research provides a highly modular architectural frame-

work for integrating mathematical statistics with Large Language Models in cybersecurity, substantially fortifying the theoretical repository concerning LLM cognitive security, guardrails, and context isolation. Practically, the finalized SENTINEL prototype serves as a deployable core engine or an independent auxiliary tool for Small and Medium Businesses (SMBs) that lack the capital to sustain large-scale Tier-1 SOC analyst teams. By operating entirely offline, the system strictly maintains data sovereignty and adheres to Zero Trust privacy guidelines.

6.3 Existing Limitations

Despite the system's demonstrated superiority, this research acknowledges several key limitations that define its operational boundary. Regarding input log formats, the implementation was evaluated primarily on Network Flow logs and HTTP payloads, whereas real-world enterprise environments require the ingestion of diverse unstructured formats—such as Windows Event Logs, Linux Syslogs, and CloudTrail records—demanding more complex semantic parsing layers. In terms of hardware constraints, although inference speed is accelerated using semantic caching, analyzing entirely novel alerts still requires substantial LLM processing cycles, mandating significant capital expenditures for dedicated GPU hardware if deployed at high-throughput network scales. Finally, regarding physical mitigation design, the research deliberately restricted the agent to generating recommended actions—such as proposing firewall rule updates—rather than executing automated physical blocks on network routers. While this constraint prevents the risk of accidental Self-Denial of Service (Self-DoS), it limits the system's capacity for fully autonomous, real-time blocking.

6.4 Future Development Directions

Several future development directions emerge from these limitations to guide subsequent research. First, the integration of federated learning could expand SENTINEL into a decentralized architecture, enabling multiple SOC endpoints to share learned threat parameters and model weights regarding malware mutations without transmitting privacy-violating raw network logs. Second, the monolithic LangGraph agent could be decomposed into a Multi-Agent System (MAS) comprising specialized sub-agents, such as malware, network, and threat intelligence analysts, which debate the evidence before issuing a final verdict to enhance classification accuracy. Third, the posture of the system could be evaluated by constructing an automated Red Team agent to continuously generate sophisticated adversarial payloads targeting SENTINEL, facilitating autonomous self-evaluation and defensive upgrades.

*Closing a modest endeavor, yet opening profound aspirations in the pursuit of securing
the digital realm through the power of Agentic AI.*

References

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Dataset for Cyber Security in Ad hoc Networks,” in *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [2] A. Alsubaei, A. Abuhussein, and S. Shiva, “Security Information and Event Management (SIEM) Features, Challenges, and Solutions,” *IEEE Access*, vol. 9, pp. 121111–121128, 2021.
- [3] P. M. M. S. Silva *et al.*, “The Big Data Security Challenges in the Era of Cloud Computing,” *Computer Networks*, vol. 156, pp. 102–115, 2019.
- [4] M. Alshamrani *et al.*, “A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2021.
- [5] S. Hassan, M. A. Saleem, and O. E. Ogu, “Alert Fatigue in Cybersecurity: A Review,” *IEEE Access*, vol. 10, pp. 58632–58645, 2022.
- [6] B. P. Welford, “Note on a Method for Calculating Corrected Sums of Squares and Products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [7] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] LangChain AI, “LangGraph: Multi-Actor, Stateful LLM Applications,” *GitHub repository*, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [9] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, “More than you’ve asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models,” *arXiv preprint arXiv:2302.12173*, 2023.
- [10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Revision 2, 2012.
- [11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Bootstrap Product Algorithms,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

- [12] T. Ravindran *et al.*, “Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content,” *arXiv preprint arXiv:2605.24421*, 2026.
- [13] A. Researcher *et al.*, “CyberRAG: An Agentic RAG cyber attack classification and reporting tool,” *Future Generation Computer Systems*, vol. 176, p. 108186, 2026.
- [14] A. Abdennebi *et al.*, “LanG – A Governance-Aware Agentic AI Platform for Unified Security Operations,” *arXiv preprint arXiv:2604.05440*, 2026.
- [15] M. Analyst *et al.*, “Policy-Guided Threat Hunting: An LLM enabled Framework with Splunk SOC Triage,” *arXiv preprint arXiv:2603.23966*, 2026.
- [16] A. Vaswani *et al.*, “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [17] A. Gholami *et al.*, “A Survey of Quantization Methods for Efficient Neural Network Inference,” *arXiv preprint arXiv:2103.13630*, 2021.
- [18] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [19] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–380, 2009.